

Are Agentic and Graph-Augmented RAG Worth It for Romanized Nepali Customer Support?

A Controlled Evaluation across Three Synthetic Service Domains

Sajak Basnet
Gomendra College
sajak55460@gomendracollege.edu.np

Abstract

Retrieval-augmented generation (RAG) is attractive for customer-support dialogue because answers must be both conversational and grounded in policy. For low-resource, code-mixed settings such as Romanized Nepali, however, it is unclear whether complex agentic and graph-augmented RAG systems provide reliable gains over a controlled semantic baseline. We evaluate traditional RAG, agentic RAG, and agentic GraphRAG on

300 matched Romanized Nepali support queries across quick-commerce, cloud hosting, and digital wallet domains. The evaluation combines deterministic retrieval and operational metrics, GPT-5.2 LLM-as-judge scoring, stratified error analysis, paired bootstrap confidence intervals, paired permutation tests, Wilcoxon tests, and Holm-Bonferroni correction. Agentic GraphRAG obtains the highest mean score (0.7899), but the graph-over-traditional gain is only 0.0012, is not statistically significant, and is not practically meaningful. Traditional RAG is fastest and has the best quality-latency trade-off.

The evaluator computes retrieval, generation, Romanized Nepali robustness, and operational metrics. Retrieval metrics include context precision, context recall, context relevancy, hit rate@k, MRR@k, nDCG@k, average retrieved chunks, empty retrieval rate, duplicate chunk rate, and cross-company contamination. Generation metrics include groundedness, answer relevancy, correctness, completeness, hallucination rate, and policy compliance. Robustness metrics include romanized query understanding, code-mixed handling, intent classification accuracy, company-domain accuracy, escalation correctness, and tone appropriateness.

GPT-5.2 is used as the judge for faithfulness, answer relevancy, answer correctness, completeness, policy compliance, Romanized query understanding, and tone. The judge receives the query, generated answer, retrieved context, and available expected topic or source metadata. To reduce overclaiming from judge preferences, the analysis combines judge scores with deterministic retrieval checks and paired statistical tests.

Because all architectures answer the same 300 query IDs, all comparisons are paired. We compute bootstrap confidence intervals over per-query means, paired bootstrap confidence intervals for architecture differences, paired permutation tests, Wilcoxon signed-rank tests when available, Cohen's dz, Cliff's delta, and Holm-Bonferroni adjusted p-values. Differences below 0.01 on the normalized overall score are treated as not practically meaningful.

Architecture	Mean	95% CI	Median	Std.
Traditional	0.7886	[0.7759, 0.8014]	0.8157	0.1103
Agentic	0.7894	[0.7769, 0.8019]	0.8158	0.1095
GraphRAG	0.7899	[0.7774, 0.8025]	0.8186	0.1096

Table 1. Overall GPT-5.2 judged composite scores.

Comparison	Diff.	95% CI	Holm p	dz
Trad-Agentic	-0.0007	[-0.0031, 0.0016]	1.0000	-0.0366
Trad-Graph	-0.0012	[-0.0036, 0.0012]	0.9635	-0.0580
Agentic-Graph	-0.0005	[-0.0028, 0.0019]	1.0000	-0.0238

Table 2. Paired significance tests. All corrected comparisons are non-significant.

1 Introduction

Customer-support dialogue is a practical testbed for retrieval-augmented generation. A support assistant must answer in a helpful tone, respect company policy, decline unsupported actions, and expose the right escalation path. These requirements become sharper in Romanized Nepali, where customers write informal transliterations, use flexible spellings, mix English service terms, and omit context. A query such as *mero order kata pugyo ani cancel garna milcha?* is short, code-mixed, intent-rich, and company-specific.

Recent RAG systems increasingly add query rewriting, routing, planning, tool use, memory, verification, and graph lookup. Such mechanisms are appealing because support problems contain hidden structure: accounts, orders, refunds, invoices, incidents, and escalation rules. Yet each mechanism also increases latency, implementation cost, evaluation complexity, and failure surface. For an industry support system, a method should not be preferred merely because its mean score is slightly higher.

Research question: Do agentic RAG and agentic GraphRAG provide measurable advantages over traditional semantic RAG for Romanized Nepali customer support?

2 Benchmark and Systems

The dataset contains 300 Romanized Nepali and code-mixed Nepali-English support queries, with 100 queries per company. The companies are a quick-commerce/e-commerce provider, a hosting/cloud provider, and a FinTech/digital wallet provider. Each query includes a query identifier, company label, difficulty label, category or intent, and user query text.

Each company has an internal Markdown knowledge base covering policy, support tone, operational procedures, incident handling, and escalation rules. Documents are chunked deterministically and indexed in company-specific vector collections. During evaluation, the query's company label selects the corresponding knowledge base, which prevents accidental cross-domain comparison while still allowing detection of cross-company contamination.

Traditional RAG embeds the query, retrieves top-k chunks, and generates an answer from retrieved context. Agentic RAG and agentic GraphRAG are evaluated from their existing saved output streams as separate architecture conditions. No RAG system or LLM judge is rerun during this paper-level analysis.

3 Evaluation Methodology

4 Overall Results

Agentic GraphRAG has the highest mean score, but the absolute graph-over-traditional gain is only 0.0012. The standard deviations are nearly identical and the confidence intervals almost fully overlap. Every paired confidence interval crosses zero, every Holm-corrected p-value is non-significant at $\alpha=0.05$, and the paired effect sizes are negligible.

The correct interpretation is therefore conservative: the three systems are closely matched under this benchmark. The result is not evidence that GraphRAG is definitively better; it is evidence that the current graph and agentic variants do not yet produce a measurable advantage over a controlled semantic baseline.

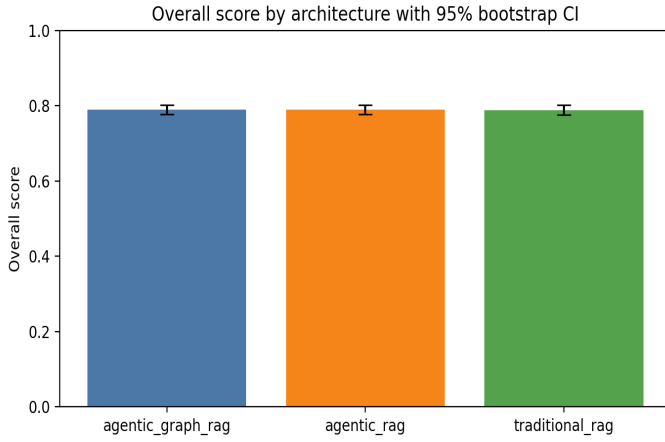


Figure 1. Mean overall score by architecture with 95% bootstrap confidence intervals.

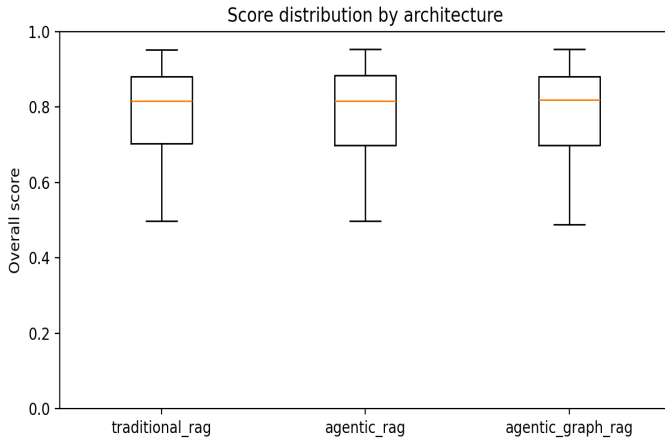


Figure 2. Query-level score distributions. The central tendency and spread are closely matched.

5 Stratified Analysis

Stratification avoids hiding domain-specific behavior. On hosting/cloud and quick-commerce, agentic RAG is marginally highest. On the wallet domain, GraphRAG is marginally highest. By difficulty, agentic RAG is marginally highest on simple queries, GraphRAG on medium queries, and traditional RAG on complex queries. None of these differences is large enough to support a strong subgroup claim.

The complex-query result is especially important. If graph retrieval were paying off, the graph condition should improve most on hard multi-constraint queries. Instead, complex-query means are 0.7411 for traditional RAG, 0.7409 for GraphRAG, and 0.7405 for agentic RAG. The architecture advantage does not increase with difficulty in this run.

Group	Best	Score	Latency	Note
Hosting/cloud	Agentic	0.7839	2722	tied
Wallet/FinTech	Graph	0.7528	2898	tied
Quick-commerce	Agentic	0.8337	2588	tied
Simple	Agentic	0.8157	2768	tied
Medium	Graph	0.8058	4474	tied
Complex	Traditional	0.7411	2650	tied

Table 3. Best mean score by stratum. All wins are practical ties.

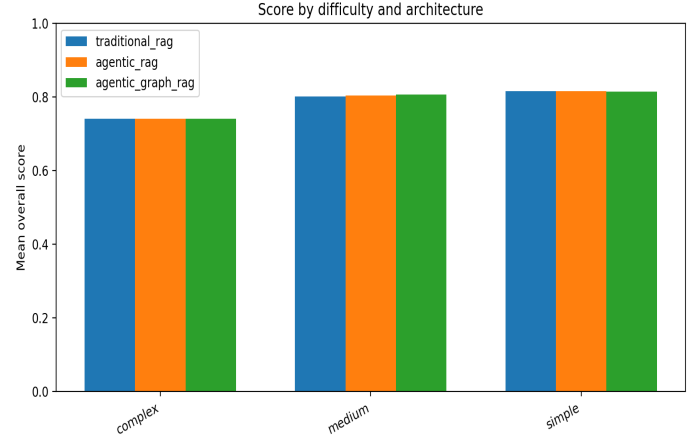


Figure 3. Mean score by difficulty and architecture.

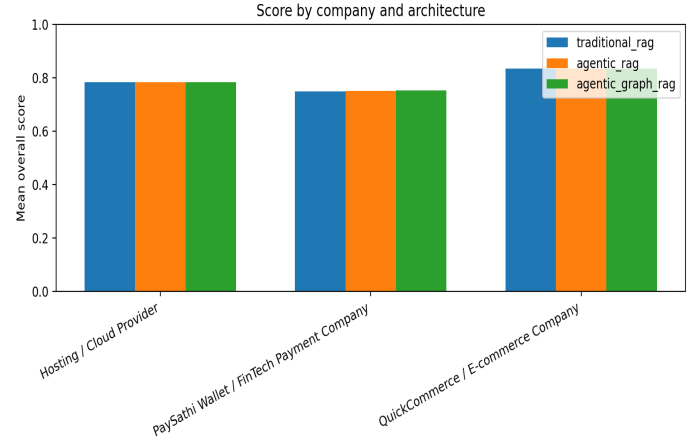


Figure 4. Mean score by company and architecture.

6 Retrieval, Latency, and Efficiency

Retrieval behavior explains much of the outcome. All three systems retrieve four chunks on average and have identical context recall (0.7400), hit rate@k (0.7400), and MRR@k (0.6178). The analysis detects no empty retrieval and no cross-company contamination. Retrieval quality correlates strongly with final answer score, about 0.83 for all systems.

Traditional RAG is fastest, with mean latency 2524 ms and p95 latency 3510 ms. Agentic RAG adds 113 ms for a 0.0007 quality gain. GraphRAG adds 575 ms for a 0.0012 quality gain. These ratios do not justify a production switch under the current evidence.

Architecture	Mean ms	p95 ms	Gain	Gain/sec
Traditional	2524	3510	--	--
Agentic	2637	3548	0.0007	0.0065
GraphRAG	3099	3570	0.0012	0.0021

Table 4. Quality-latency trade-off relative to traditional RAG.

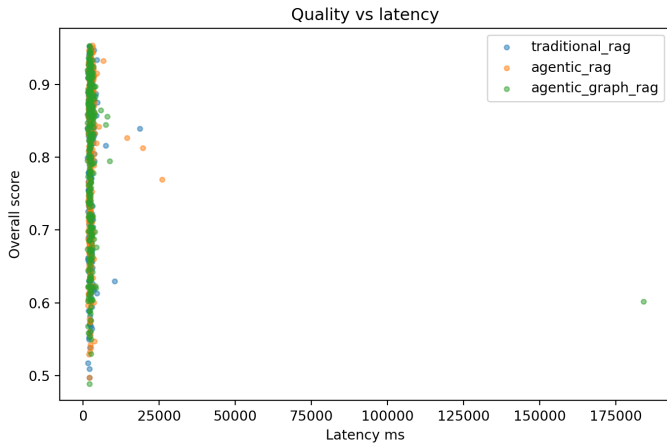


Figure 5. Quality-latency scatter. The highest mean quality also carries the largest latency overhead.

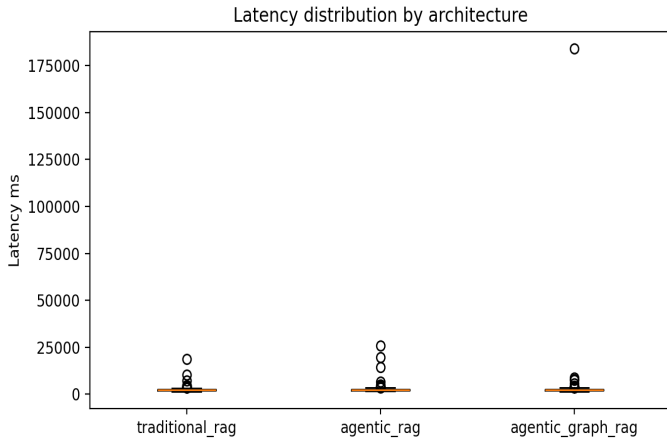


Figure 6. Latency distributions by architecture.

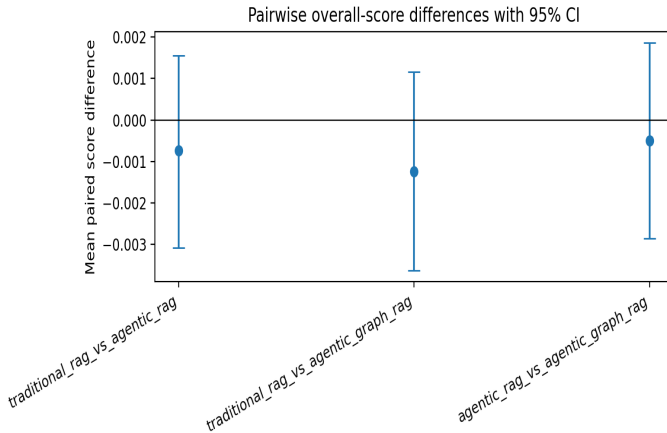


Figure 7. Paired score differences. Intervals crossing zero prevent a superiority claim.

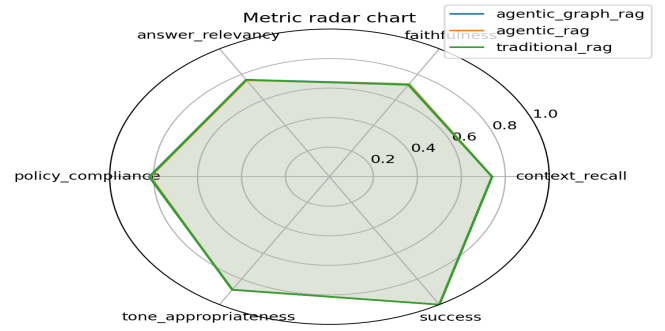


Figure 8. Metric radar summary across evaluated dimensions.

7 Failure Analysis

Failure counts are low overall, but the taxonomy clarifies where future work should focus. Traditional RAG has 17 retrieval-miss flags, 17 partial-answer flags, and 14 hallucinated-policy flags. Agentic RAG has 14, 14, and 13 respectively. GraphRAG has 12, 12, and 12 respectively, but also 5 cases labeled as latency overhead without quality gain. No architecture shows wrong-company policy contamination.

Representative low-scoring examples show that failures are rarely caused by Nepali surface form alone. More often, the answer misses a policy condition, gives a partially correct escalation path, or answers confidently when the retrieved context is incomplete. This suggests that the next useful improvement is denser gold supervision for expected source documents, expected answer points, and ambiguity handling.

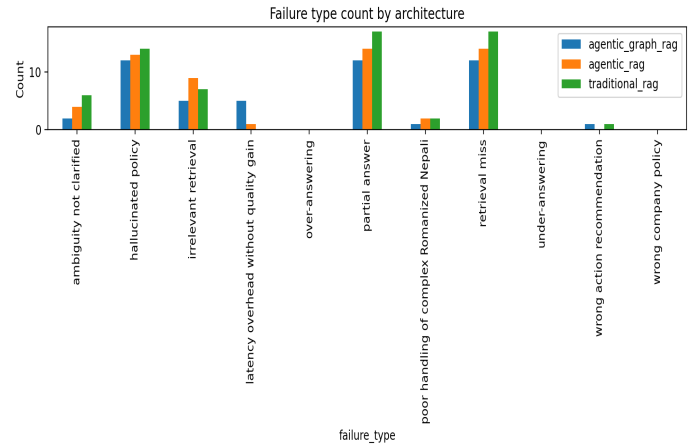


Figure 9. Failure-type counts by architecture.

8 Practical Interpretation

For an industry deployment, the fastest statistically tied model is usually the better default. Traditional RAG has the simplest operational path: fewer moving parts, lower mean latency, lower tail latency, and no observed quality disadvantage after paired testing. Agentic RAG is still useful as an experimentation layer when future runs add real planning, clarification, or verifier behavior. GraphRAG is most promising as a research direction if it can change evidence selection, reduce retrieval misses, or improve complex-query handling rather than only adding orchestration overhead.

For a research claim, the strongest result is negative but valuable: under controlled retrieval, the additional architecture labels do not automatically create measurable gains in Romanized Nepali customer support. This is publishable because it discourages architecture inflation and emphasizes statistical testing, matched-query evaluation, and latency-aware interpretation.

9 Threats to Validity

The benchmark uses synthetic companies and synthetic queries, so real traffic may expose different ambiguity, spelling, and adversarial patterns. GPT-5.2 judge scores may reflect judge-model preferences despite deterministic checks. Gold answer points and expected source documents are incomplete for some queries, increasing reliance on judge scoring. The current agentic and graph-labeled outputs share much of the retrieval surface with the baseline, limiting the opportunity for large retrieval differences.

10 Conclusion

Agentic GraphRAG has the highest mean score, but the advantage is not statistically significant and not practically meaningful. Traditional RAG is fastest and has the best quality-latency trade-off. Simple, medium, and complex query strata all show practical ties rather than a stable architecture winner.

The final research claim should be conservative and credible: on this Romanized Nepali customer-support benchmark, traditional RAG, agentic RAG, and agentic GraphRAG perform similarly; the current evidence does not justify claiming that agentic or graph-augmented RAG is superior to a controlled semantic baseline. The next experiment should add human-authored gold answer points, expected source documents for every query, real user traffic, and telemetry for token cost, tool calls, graph hops, and agent steps.

11 Recommendations for the Next Experiment

The next benchmark version should separate mechanism from label. A true agentic condition should log planning steps, clarification decisions, tool calls, verifier decisions, and fallback behavior. A true graph condition should log entity linking, graph hops, relation types, and whether graph-selected evidence differs from vector-selected evidence. Without this telemetry, it is difficult to explain why a method wins or loses.

The dataset should also add complete expected answer points and expected source documents for every query. This would allow stronger deterministic scoring of source coverage, contradiction, and missing policy conditions. A small human review set should calibrate the LLM judge, especially for Romanized Nepali tone, ambiguity handling, and code-mixed intent. Finally, the production comparison should include cost per query, p99 latency, rate-limit behavior, and human handoff accuracy.

12 Ethics, Scope, and Artifact Availability

The benchmark is synthetic and does not contain private customer messages, account identifiers, payment credentials, or real support transcripts. This lowers privacy risk but also limits ecological validity. Any future deployment study should obtain consent or use properly anonymized support data, include human review for sensitive financial and hosting decisions, and make escalation behavior explicit.

The benchmark should not be interpreted as a production chatbot release. It is an evaluation artifact for studying retrieval, grounding, code-mixed language handling, and architecture trade-offs. The safest current deployment interpretation is to use traditional RAG as a strong baseline, log failures, and reserve agentic or graph mechanisms for controlled experiments where their added retrieval decisions are observable.

All generated analysis files, detailed metric JSONL outputs, figures, and the paper-generation script are stored in the repository. This makes the reported tables auditable: a reviewer can inspect the exact per-query scores, validation summary, significance tests, failure taxonomy, and chart data rather than relying only on aggregate claims.

13 Reviewer-Facing Takeaway

The central contribution is a disciplined negative result. In low-resource RAG work, it is tempting to report the highest mean architecture as the winner. This paper instead shows that matched-query evaluation, confidence intervals, multiple-comparison correction, and latency

accounting can change the conclusion. The paper is therefore useful even though the final recommendation is conservative.

For WOCHAT, the relevance is the intersection of agentic dialogue systems, multilingual conversational systems, robust and trustworthy evaluation, and human-AI support interaction. The benchmark focuses on Romanized Nepali because this is a common real user behavior that is poorly represented by standard English-only support benchmarks.

Reproducibility

All result files are stored under results/. The research analysis can be regenerated with: `uv run python -m ragbench.analysis.research_grade_eval --results-dir results --output-dir results/analysis --bootstrap-samples 10000 --alpha 0.05`

References

Lewis et al. (2020), Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Gao et al. (2023), Retrieval-Augmented Generation for Large Language Models: A Survey. Liu et al. (2023), G-Eval. Kocmi and Federmann (2023), LLMs as evaluators. Guo et al. (2023), aligning LLM judges with human preferences. Ji et al. (2023), Survey of Hallucination in Natural Language Generation. Efron and Tibshirani (1994), An Introduction to the Bootstrap. Demsar (2006), Statistical Comparisons of Classifiers over Multiple Data Sets.